AI Engineering Harness — Walkthrough Presentation

Serge Kauzunovich | Intrahealth Interview Task | 2026-04-16 React / Medplum | Format: Screen-share walkthrough | Total time: ~18 minutes **Harness repo:** https://github.com/SergeKauzunovich/intrahealth-harness-react **Work order (Issue #1):** https://github.com/SergeKauzunovich/intrahealth-harness-react-test/issues/1

Slide 1 — The Brief in One Sentence *(~1 min)*

> *"We're not asking you to build a drug interaction checker.*
> *We're asking you to build the **machinery that builds it**."*

The drug interaction checker is the *work order*. The harness is the point.

What I built:

- An autonomous React/Medplum harness that takes a GitHub Issue and produces a PR
- The work order: `DrugInteractionPanel` — a clinician-facing interaction review component
- Built on a single architectural insight: **Claude Code IS the harness**

Open: https://github.com/SergeKauzunovich/intrahealth-harness-react **Issue #1 (work order):** https://github.com/SergeKauzunovich/intrahealth-harness-react-test/issues/1

Slide 2 — The Core Insight *(~1 min)*

Most AI harness designs look like this:

```
Your code → calls Claude API → gets code back → your code validates it
```

This requires maintaining an orchestration codebase that you update as models improve.

What I built instead:

```
Claude Code IS the orchestrator
  settings.json hooks      → enforcement at every tool call
  /harness command         → pipeline entry point (markdown instructions)
  react-ui-builder agent   → domain specialist (React/Medplum)
  /judge command           → completeness evaluator (fresh context)
```

The entire harness is **text files**. When models improve, you update the markdown. No TypeScript to refactor. No API wrapper to maintain. The harness is "rippable" by design.

Slide 3 — End-to-End Flow *(~1.5 min)*

```
GitHub Issue labeled 'ai-agent'
        │
        ▼
claude "/harness <issue-url>"
        │
        ▼
/harness (harness.md):
  ├─ Fetch issue body
  │
  ├─ ⛔ STOP 1: present requirements understanding → human confirms
  │
  ├─ Load L1 + L2 context + observations/
  ├─ Draft implementation plan → saved to implementation-plans/issue-N.md
  │
  ├─ ⛔ STOP 2: show plan → human confirms before any code is written
  │
  ├─ git worktree add worktrees/issue-N -b feature/issue-N
  ├─ Write .claude/current-worktree (scopes hooks to this branch only)
  ├─ Agent(react-ui-builder) ── works autonomously in worktree
  │       hooks fire on every tool call (enforcement layer)
  │       runs npm build/test itself; self-corrects on failures
  ├─ Agent(improvement-agent) ── captures self-corrections → observations/
  ├─ /code-annotator ── adds JSDoc to all exported TypeScript symbols
  ├─ /judge ── fresh context, reads diff, runs tests, returns JSON
  │       if incomplete → retry with gap list (max 3×)
  │
  ├─ ⛔ STOP 3: show branch, base, file list → human confirms before commit
  │
  └─ /create-pr ── commit + push + gh pr create + worktree remove
```

Walk the repo: show `.claude/` directory, open `harness.md`, open `settings.json`.

Slide 4 — The Four Areas *(transition slide)*

The brief evaluates on four areas. Each has a concrete answer in this design.

```
1. Harness Architecture    → Claude Code native: hooks + commands + agents
2. Task Decomposition      → The GitHub Issue IS the work order (show issue-1.json)
3. Quality Gates           → npm build → test → lint, self-correcting agent, judge + retry
4. Failure Modes           → hooks block bad actions; JSONL captures everything
```

Slide 5 — Area 1: Harness Architecture *(~3 min)*

Orchestration

What kicks off the agent work?

GitHub Actions fires on `issues: [labeled]` when `label.name == 'ai-agent'`.

```
# .github/workflows/harness.yml
- name: Run harness
  run: claude --print "/harness ${{ github.event.issue.html_url }}"
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

`claude --print` runs Claude Code headlessly. `/harness` is defined in `.claude/commands/harness.md` — plain English instructions that Claude follows as a pipeline.

Show file: `.claude/commands/harness.md` (10-step pipeline in markdown)

Context delivery

How does the agent understand a Medplum codebase it's never seen?

3-layer progressive disclosure — the agent never gets the whole SDK dumped into its context.

```
L1 (always):   CLAUDE.md (≤200 lines) + context-map.md (50 lines)
L2 (targeted): pattern docs the agent selects from the map
L3 (on-demand): full file content when agent explicitly reads a file
```

The key constraint: `CLAUDE.md` must stay under 200 lines. It defines the project contract: directory layout, naming conventions, the `DrugInteractionPanelProps` signature (immutable), scaffold commands, quality gate commands, and 5 rules.

`context-map.md` is a navigational index:

```
medplum-types.md         → Read if: implementing component or hooks
ui-builder-example.md    → Read if: unsure about file layout or structure
react-testing-patterns.md → Read if: writing tests or setting up Vitest
mock-fhir-data.md        → Read if: implementing mock data or test scenarios
```

The agent reads the map, decides what it needs, pulls only that. Context stays lean even as Medplum's SDK grows.

Show files: `harness-context/context-map.md` and `CLAUDE.md`

Code generation

Where does the agent put its code?

In a git worktree — an isolated branch created per issue:

```
git worktree add worktrees/issue-1 -b feature/issue-1 master
```

The `react-ui-builder` agent is a specialist persona. It knows the implementation order: types → mock data → test setup → hook → sub-components → main component → tests.

It uses Claude Code's native tools (Read, Write, Edit, Bash, Glob) — no custom tool wrappers needed. The hooks on those tools enforce all boundaries.

Show file: `.claude/agents/react-ui-builder.md`

PR assembly

What's in the PR?

`/create-pr` reads the JSONL log and assembles:

- Gate attempt count (how many npm build/test cycles were needed)
- Judge assessment (COMPLETE or GAPS)
- Known gaps (from judge.concerns)
- Link to the JSONL log for full replay

```
## Quality gates
- [x] npm run build (TypeScript strict)
- [x] npm test -- --run
- [x] npm run lint

## AI generation metadata
- Gate attempts: 2
- Judge assessment: COMPLETE
- Run log: logs/issue-1.jsonl

## Known gaps
None
```

Show file: `.claude/commands/create-pr.md`

Human checkpoints

The harness has three mandatory stops — the human must confirm before the pipeline advances.

Checkpoint 1 — Requirements understanding: After reading the issue, the harness summarises its interpretation: what to build, key acceptance criteria, the immutable interface contract, and any constraints it noted. The human confirms or corrects before any context loading or planning begins.

Checkpoint 2 — Implementation plan: After loading all context, the harness drafts a plan (file structure, implementation order, key decisions) and saves it to `implementation-plans/issue-N.md`. The human reviews the plan before any code is written or any worktree is created.

Checkpoint 3 — Pre-commit review: After the judge says COMPLETE, the harness presents the proposed branch name, base branch, and a full `git diff --stat`. The human confirms (or cancels, leaving the branch intact for manual review) before any commit, push, or PR is created.

These are deliberate gates — not retries, not automatic. No code reaches GitHub without a human seeing the plan and the diff.

Show file: `.claude/commands/harness.md` — show the ⛔ STOP blocks

Slide 6 — Area 2: Task Decomposition *(~2 min)*

The GitHub Issue is not just a description — it **is the work order**.

What makes a good agent ticket?

The brief asks: *"Too vague and the agent hallucinates. Too prescriptive and you've written the code yourself. Where's the line?"*

My answer: **specify the contract, leave the implementation.**

```
✓ Specify:  The component props interface (DrugInteractionPanelProps — immutable)
✓ Specify:  The four required visual states with exact data-testid values
✓ Specify:  The API contract (POST /api/drug-interactions/check, 200/503)
✓ Specify:  Acceptance criteria as checkboxes (machine-evaluable by /judge)
✗ Don't:    Tell the agent how to implement useInteractionCheck()
✗ Don't:    Write the test bodies in the ticket
```

Show file: `tests/fixtures/issue-1.json` — point out:

- Typed props interface prevents hallucinated signatures
- `data-testid` values specified → judge can verify each state mechanically
- Architecture constraints: `useInteractionCheck` hook owns all fetching, component stays thin

Domain knowledge: Where did the FHIR interaction scenarios come from?

I consulted an `alex-paramedic` agent (a Claude sub-agent with 10+ years Toronto paramedic experience) to get clinically accurate scenarios — mechanisms, severity tiers, real patient risk factors. This became `mock-fhir-data.md`: 3 concrete patient scenarios.

Show file: `harness-context/examples/mock-fhir-data.md` — the 3 FHIR scenarios

The separation of concerns principle applies to tickets and context docs too:

- Ticket specifies WHAT (acceptance criteria, interface contract)
- CLAUDE.md specifies WHERE (directory layout, naming conventions)
- Pattern docs specify HOW (TypeScript types, component skeletons, test templates)

Three separate documents. Each has a clear scope. The agent reads only what it needs.

Slide 7 — Area 3: Quality Gates *(~2 min)*

What stands between "the agent wrote code" and "a PR is opened for human review"?

Three layers:

Layer 1: Self-correcting agent

The `react-ui-builder` agent is instructed:

> *"Do not signal completion until `npm run build` AND `npm test -- --run` both pass. Run them yourself. If they fail, read the full error output and fix it. Keep going."*

This creates a self-correction loop with no explicit retry counter in the harness. A typical run:

```
Build attempt 1: TS2322 type mismatch → agent reads error, fixes type → build passes
Test attempt 1:  waitFor timeout — data-testid not found → agent fixes component → tests pas
Lint:            Auto-fixed by format-ts-files.sh hook on every .tsx edit
```

Layer 2: The judge

After the agent returns, `/judge` runs in a **completely fresh context** — no memory of how the code was generated. It reads the diff, runs tests independently, checks the 14-point acceptance criteria checklist:

```
{
  "complete": true,
  "missing": [],
  "concerns": ["acknowledge button only shown for major severity, not moderate"]
}
```

`concerns` are non-blocking observations that appear as "Known gaps" in the PR — giving the human reviewer exactly what to verify.

Layer 3: Retry with gap context

If `complete: false`, the harness re-runs the agent with:

> *"Known gaps from previous attempt: [judge.missing]. Address all of these."*

Maximum 3 attempts. After 3 failures: `needs-human-review` label + draft PR (work preserved, nothing discarded).

Human handoff

Show file: `.claude/commands/judge.md` — walk through the 14-point checklist

The PR gives the reviewer:

1. Which gates passed/failed and how many cycles
2. What the judge flagged as missing or concerning
3. The full JSONL log for decision replay

Slide 8 — Area 4: Failure Modes & Observability *(~2 min)*

Context limits

Prevention: `CLAUDE.md` held to ≤200 lines. Progressive disclosure via the context map means the agent never loads Medplum's full type definitions — only the pre-extracted subset relevant to this component.

Detection: If the agent produces confused output, `/judge` catches it as `complete: false`. The retry uses a slimmer context (L1 only, no L2 pattern docs).

Hallucination

Detection: TypeScript strict mode (`npm run build`) catches hallucinated APIs immediately:

- `TS2339` : property doesn't exist on Medplum type
- `TS2304` : cannot find name (invented hook or component)

The agent gets the full TypeScript error output and self-corrects. Broken TypeScript cannot pass the build gate — it cannot silently slip through.

Tangent detection

Prevention via hooks: `validate-write-path.sh` fires on every `Write` tool call. If the path is outside the current worktree, the tool is blocked before the write happens:

```
BLOCKED: write to '/some/other/file.tsx' is outside the current worktree
```

The agent cannot touch files outside its assigned branch. Scope is enforced at the infrastructure level, not by trust or instruction alone.

Show file: `.claude/hooks/validate-write-path.sh` — show the path comparison logic

Stuck detection

3 failed `/judge` evaluations → issue labeled `needs-human-review` + draft PR created. Work is never discarded — the incomplete component is committed so a human can review and continue where the agent left off.

Observability

Every quality gate is logged by `log-bash-output.sh`:

```
{"ts":"2026-04-16T15:01:12Z","stage":"gate","cmd":"npm run build","exit_code":0}
{"ts":"2026-04-16T15:01:28Z","stage":"gate","cmd":"npm test -- --run","exit_code":1}
{"ts":"2026-04-16T15:03:45Z","stage":"gate","cmd":"npm test -- --run","exit_code":0}
{"ts":"2026-04-16T15:03:49Z","stage":"gate","cmd":"npm run lint","exit_code":0}
{"ts":"2026-04-16T15:03:51Z","stage":"stop","message":"Claude Code session ended"}
```

From this log you can replay: how many cycles ran, which failed, and how long each took. The PR description references this log and includes the gate attempt count.

Show file: `.claude/hooks/log-bash-output.sh`

Slide 9 — The Hooks: Enforcement at the Tool Level *(~1.5 min)*

This is what separates this design from a traditional retry loop.

```
Traditional approach:              This harness:

  agent writes code                  agent writes code

  harness checks output              hook fires on EVERY Write

  harness retries if bad             hook blocks if path is outside worktree

                                     hook fires on EVERY Bash

                                     hook blocks if command is destructive

                                     hook logs after EVERY npm build/test

                                     hook formats after EVERY .ts/.tsx edit
```

Enforcement is **continuous and at the tool-call level**, not a periodic batch check.

settings.json wiring:

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Write", "hooks": [{ "command": "bash .claude/hooks/validate-write-pa
      { "matcher": "Bash",  "hooks": [{ "command": "bash .claude/hooks/block-destructive
    ],
    "PostToolUse": [
      { "matcher": "Bash", "hooks": [{ "command": "bash .claude/hooks/log-bash-output.sh
      { "matcher": "Edit", "hooks": [{ "command": "bash .claude/hooks/format-ts-files.sh
    ],
    "Stop": [{ "command": "bash .claude/hooks/on-stop.sh" }]
  }
}
```

5 hooks. 5 shell scripts. All tested: `bash tests/test-hooks.sh → 11 passed, 0 failed`

Show file: `tests/test-hooks.sh` — show the test structure, then run it live

Slide 10 — The Medplum Context Challenge *(~1.5 min)*

The brief specifically calls this out for the React track:

> *"Medplum's React SDK is well-documented but non-trivial. How does your harness give agents enough context about existing component patterns, theming, and data hooks without flooding the context window?"*

The problem: If you feed the agent the whole Medplum repo, you burn the context window before it writes a line of code. If you feed it nothing, it hallucinates hook signatures.

My answer: Pre-extract exactly what this component needs. Four targeted docs:

`medplum-types.md` — Complete TypeScript interfaces for `DrugInteractionPanelProps`, `Interaction`, `InteractionCheckResponse`, the FHIR `MedicationRequest` shape, and the two Medplum hooks the agent needs (`useMedplum`, `useSearchResources`) with exact signatures. The agent never needs to open `node_modules/@medplum/react`.

`ui-builder-example.md` — Skeleton implementations of `DrugInteractionPanel.tsx` and `useInteractionCheck.ts`. The agent knows exactly where to put files and what structure to follow before writing the first line.

`react-testing-patterns.md` — Complete test file template: MSW server setup, `renderWithMedplum()` wrapper, and 5 test cases ready to adapt. The agent doesn't discover MSW patterns — it receives them.

`mock-fhir-data.md` — Three concrete patient scenarios as TypeScript objects ready to paste into `src/mocks/mockData.ts`. No FHIR expertise needed.

Total L2 context: ~400 lines. Lean enough to fit comfortably in a single context window alongside the component implementation.

Show file: `harness-context/patterns/medplum-types.md` — highlight the hook signatures

Slide 11 — Self-Improving Harness *(~1.5 min)*

The problem with static harnesses

Every new stack, every new SDK version, every new ESLint rule produces setup failures. A static harness makes the agent rediscover the same problems on every run.

The improvement loop

After each implementation the `improvement-agent` runs with a clear mandate: inspect what the react-ui-builder had to fix and write an observation file for each systemic problem.

The react-ui-builder ends its completion report with a structured table:

```
## Self-corrections applied


| Error encountered              | Root cause                        | Fix applied
|--------------------------------|-----------------------------------|----------------------
| Cannot find @mantine/core      | Undeclared peer dep of @medplum/react | npm install @manti
| 'test' not in UserConfigExport | defineConfig from vite not vitest  | import from vitest/c
```

The improvement-agent reads this table. For each row it either creates a new `.md` file in `observations/` or updates an existing one. One file = one problem category.

On the **next run**, the harness loads all observation files and passes them to the agent before it writes the first install command. The agent reads the **Prevention** sections and applies them upfront — the problem never occurs again.

```
Run 1: 5 setup problems → 5 observation files created
Run 2: agent applies all prevention steps → 0 setup problems
Run 3: new problem → 1 new observation file → 0 on run 4
```

No human curation. The harness learns automatically from every execution.

/code-annotator

After the implementation agent finishes, `/code-annotator` adds JSDoc to every exported TypeScript symbol — interfaces, type aliases, hooks, components, MSW handler arrays. The PR always ships with documented code, without the agent needing to think about it.

Show directory: `observations/` — 5 files from the first implementation run.

Slide 12 — Design Decisions *(~1.5 min)*

Why Claude Code native over a TypeScript orchestrator?

A TypeScript harness (`harness.ts`) is model-agnostic and gives tighter control over retry logic. This design wins on *rippability*: as models improve, you update markdown, not code. The brief's own reference doc says: "Build your harness to be easily removed or updated." All-text configuration is the most rippable design possible.

Why progressive disclosure over full context injection?

50-line index + 4 targeted docs (~400 lines) vs. dumping all Medplum docs at once. Even a curated Medplum subset would exceed what's needed for this one component. The context map forces the agent to think about what it needs — which itself is a useful forcing function for incremental, focused implementation.

Why hooks over an explicit retry loop?

Hooks enforce boundaries *continuously* and *before* failures compound. The write-path hook prevents an entire class of bugs (tangent edits) from ever reaching the quality gate. A retry loop catches failures after the fact; hooks stop them from happening.

Why a separate judge agent?

Producer and judge must be independent. A generator that's been wrestling with a problem for 20 tool calls may rationalize incomplete output as "good enough." The judge runs in a fresh context with zero memory of the generation session — it can only evaluate what exists.

Why worktrees over feature branches?

Worktrees let three GitHub Actions jobs run simultaneously, each with its own isolated filesystem state, with no locking or coordination. The hook reads `.claude/current-worktree` for path enforcement — each job's worktree path is different, so isolation is automatic.

Slide 13 — Live Demo *(~1.5 min)*

Walk through:

1. https://github.com/SergeKauzunovich/intrahealth-harness-react — harness repo
2. https://github.com/SergeKauzunovich/intrahealth-harness-react-test/issues/1 — Issue #1, `ai-agent` label applied
3. `.claude/` directory — settings.json, hooks/, commands/, agents/
4. `CLAUDE.md` — note ≤200 lines, the `DrugInteractionPanelProps` interface at top
5. `harness-context/context-map.md` — the progressive disclosure index
6. `tests/fixtures/issue-1.json` — the work order with checkboxes `/judge` evaluates
7. `.claude/agents/react-ui-builder.md` — highlight the key instruction:

> *"Do not signal completion until `npm run build` AND `npm test -- --run` both pass. Run them yourself and self-correct on failures."*

8. Run live: `bash tests/test-hooks.sh` → 11 passed, 0 failed

The GitHub Actions trigger is live. Applying the `ai-agent` label to Issue #1 would fire the workflow and run the full harness pipeline.

Slide 14 — Known Gaps & What I'd Add *(~30 sec)*

Currently missing:

- Cost tracking in JSONL — hooks don't expose Claude API token counts from `PostToolUse`; would need the `Stop` hook to read session metadata when it becomes available
- Streaming gate output — current JSONL is written at checkpoints, not line-by-line
- Sub-agent parallelism — e.g., spawn `useInteractionCheck` hook and tests in parallel sub-agents, merge results before PR

With more time:

- Live end-to-end run producing the actual `DrugInteractionPanel` PR
- Cost per run visible in PR description
- A second issue showing the harness handling a failing test (observing the retry loop)
- Cross-stack observation sharing — observations from the React harness informing a future Vue or Angular harness that shares the same Medplum SDK

Slide 15 — Summary *(~30 sec)*

The Four Areas, answered:

Area	Answer
Harness Architecture	Claude Code commands + hooks + specialist agent; worktrees for parallel isolation
Task Decomposition	Structured GitHub Issue: contract specified, implementation left to agent; FHIR context pre-extracted
Quality Gates	npm build → test → lint; self-correcting agent instruction; context-isolated judge; 3-attempt retry
Failure Modes	Hooks block bad writes/commands at tool level; TypeScript catches hallucinated APIs; JSONL replay

Plus two features that compound over time:

- **Self-improving harness** — improvement-agent turns every self-correction into an observation file; next run applies it upfront
- **Code annotation** — /code-annotator adds JSDoc to all exported TypeScript symbols before PR opens

The one-sentence design principle:

Enforce boundaries at the infrastructure level — let the agent's intelligence do the work, constrained by hooks that fire on every tool call, not by rules it can choose to ignore.

Harness repo: https://github.com/SergeKauzunovich/intrahealth-harness-react **Work order:** https://github.com/SergeKauzunovich/intrahealth-harness-react-test/issues/1 — 25 files, Issue #1 live with `ai-agent` label, 11/11 hook tests passing, 5 observations written

Appendix: Files to Show During Walkthrough

Timestamp	File	What to highlight
0:00	https://github.com/SergeKauzunovich/intrahealth-harness-react	Overall structure, README
1:30	https://github.com/SergeKauzunovich/intrahealth-harness-react-test/issues/1	`ai-agent` label, acceptance criteria checkboxes
3:00	`.claude/settings.json`	Hook wiring — 5 matchers
4:00	`.claude/hooks/validate-write-path.sh`	Path comparison logic
5:00	`.claude/hooks/log-bash-output.sh`	npm pattern match, JSONL write
6:00	`CLAUDE.md`	≤200 lines, corrected scaffold commands, DrugInteractionPanelProps
7:00	`harness-context/context-map.md`	Progressive disclosure index + observations section
8:00	`tests/fixtures/issue-1.json`	Acceptance criteria + `data-testid` values
9:00	`harness-context/examples/mock-fhir-data.md`	3 FHIR patient scenarios
10:00	`harness-context/patterns/medplum-types.md`	Hook signatures, no node_modules needed
11:00	`.claude/agents/react-ui-builder.md`	"read observations before scaffolding"; self-corrections table
12:00	`.claude/agents/improvement-agent.md`	Observation file format; what NOT to document
12:30	`observations/`	5 files from run 1 — show medplum-react-peer-deps.md
13:00	`.claude/commands/code-annotator.md`	JSDoc rules; verify with npm run build
14:00	`.claude/commands/judge.md`	14-point checklist

15:00	`tests/test-hooks.sh`	Run live: 11/11 passing